

Reinsurance Recon Utility

Product Case Study — Detailed Reference

Role: Solo Product Owner & Builder
Context: Big 4 Consulting — Global Reinsurance Finance Transformation Program
Stack: Streamlit · Pandas · PySpark · SQL
Status: Live in production · Expanding to additional reinsurance projects
Details anonymized to protect client confidentiality.

1. Executive Summary

During a large-scale reinsurance Finance Transformation program, a critical gap was identified in the processing validation cycle — reconciliation of ceded reinsurance amounts against treaty rules was being done manually in Excel, taking 2 working days per monthly cycle, and had become a dependency on technical QA resource rather than an independently operable BAU process.

A self-serve Reinsurance Recon Utility was conceived, defined, and built — a hard gate validation tool that sits between the reinsurance engine and downstream systems (subledger, GL, reporting, settlement) — ensuring cession accuracy is confirmed against treaty rules before any financial data is released downstream.

Through three iterations driven by user feedback and adoption barriers, the tool evolved from a SQL script to a fully self-serve Streamlit application operable by non-technical BAU analysts with a simple set of clicks.

Metric	Result
Recon cycle time	2 working days → 15 minutes (99.6% reduction)
BAU dependency	Technical QA dependency fully eliminated
Scale	Millions of transactions, billions in financial impact per cycle
Adoption	Expanding to additional reinsurance projects firm-wide

2. The Problem

Context

A large global insurer was undergoing an enterprise-wide Finance Transformation program — modernizing multiple platforms to enhance month-end close, eliminate reporting lag, drive automation, and standardize group companies onto a common GL. The RI Transformation stream replaced disparate homegrown systems and manual processes with a unified, automated, globally standardized solution.

At the heart of this transformation was a reinsurance engine — responsible for processing inbound gross transaction data (premiums, commissions, claims, reserves) and applying treaty rules to produce ceded, retained, and error-classified output. This output feeds directly into downstream subledgers, GL, reporting layers, and reinsurer settlement — making accuracy non-negotiable.

The Gap

Every production cycle, someone had to answer one question: *did the reinsurance engine apply the treaty rules correctly?*

The answer required reconciling actual engine output against expected cession amounts — calculated manually from treaty parameters including attachment criteria, ceding percentages, and contract configuration — for every policy, every cycle. This was being done in Excel.

- Manual recon of one month's data took **2 full working days** per cycle
- BAU analysts regularly offloaded recon work to the QA team due to volume and complexity
- The QA team became the de facto recon resource on top of their primary validation responsibilities
- Errors were caught late, after significant analyst effort had already been spent
- At scale — millions of transactions per run, billions in financial impact — this was a critical control gap

Why It Mattered

Risk	Description
Financial Misstatement	Incorrect ceded amounts appear in financial statements — regulatory and audit exposure at g
Treaty Breach	Policies ceded outside attachment criteria violate contract terms and can void reinsurer recoveries
Audit & Compliance	Manual recon with no automated control is a flagged weakness in internal and external audit reviews
Operational Cost	2 working days of senior analyst time per cycle, every month, across multiple active treaties

3. Opportunity Validation

Before building anything, every product opportunity must pass three checks. This one passed all three.

Check 1 — Strategic Fit ✓

The utility sat squarely within the RI Transformation stream's mandate — replacing manual, technically-dependent processes with automated, standardized solutions operable by BAU teams globally. It directly served four Finance Transformation Program objectives: month-end close enhancement, reporting lag elimination, automation of manual processes, and strengthening analytical capabilities for business partners.

Check 2 — User Value ✓

Three user groups were directly impacted:

BAU Operations Analysts

Non-technical users responsible for running monthly reinsurance processing cycles. Currently spending 2 working days per cycle on manual Excel recon — interpreting per-policy attachment criteria individually, with no automated support. Pain was high severity: slow, error-prone, and spilling over into QA team capacity.

QA / Data Engineers

Technical users absorbing BAU recon overflow on top of primary validation responsibilities — misallocating skilled resource to manual operational work.

Product Owners / Delivery Leads

Responsible for signing off on downstream release covering billions in financial impact. Without automated validation, sign-off relied on manual recon that was slow and difficult to audit. The utility gave POs structured,

automated evidence of cession accuracy — replacing subjective manual checks with a repeatable, documented control.

Check 3 — Business Value ✓

Dimension	Content
Qualitative	Stronger automated controls; reduced audit risk; elimination of manual recon as a control weakness
Quantitative	99.6% reduction in recon cycle time (2 days → 15 minutes); QA recon dependency eliminated
Non-Goals	Not judged on reinsurance engine performance or upstream data quality

A structured Manager Briefing was completed before any build work began. See Appendix A.

4. The Product

What It Does

The Reinsurance Recon Utility is a self-serve validation application that automatically reconciles actual reinsurance engine output against expected cession amounts — derived from treaty rules — acting as a validation gate before any financial data is released to downstream systems.

How It Works

Input	What It Contains
Treaty Report / Coding Sheet	Contract configuration — attachment criteria, ceding percentages, contract name, retention rule
Inbound File	Gross transaction data from source systems — premiums, commissions, claims, reserves
Reinsurance Engine Output	Actual processed data — ceded amounts, retained amounts, error-classified transactions

Validation Output

- Correctly ceded transactions
- Transactions present in error queue — flagged with reason
- Transactions incorrectly retained — flagged for investigation
- Cession amount mismatches — actual vs expected variance surfaced

Architecture

Source Systems (gross transaction data)
↓
Reinsurance Engine (applies treaty rules → produces ceded / retained / error output)
↓
RECON UTILITY — Treaty Rules → Expected vs Actual → Validated
↓ (BAU reviews output and releases to downstream after validation passes)
Downstream: Subledger → GL → Reporting → Settlement

Application Screenshots

The utility is built in Streamlit — a browser-based interface requiring no technical prerequisites. A BAU analyst runs a full recon cycle through a simple sequence of file uploads and clicks.

←↻🏠🕒localhost:8501

Summarize🔍🌟⚙️🔖📄👤⋮

Reinsurance Reconciliation Tool

International & Domestic RI Calculation

📁 Step 1: Upload Input Files

🌟 Required: Inbound and Coding Sheet. (Optional: Outbound, Error, Retention)

🌟 Inbound File (comma or pipe-separated, or .zip)

📄 Upload500MB per file • TXT, CSV, ZIP

🌟 Outbound File (comma or pipe-separated, or .zip)

📄 Upload500MB per file • TXT, CSV, ZIP

Retention File — optional (pipe-separated)

📄 Upload500MB per file • TXT, CSV

🌟 Coding Sheet (comma-separated .csv)

📄 Upload500MB per file • TXT, CSV

🌟 Error File (comma or pipe-separated, or .zip)

📄 Upload500MB per file • TXT, CSV, ZIP

⚙️ Company Code Configuration

Define how inbound worldwide company codes map to manager company codes (applies to both International and Domestic). Use 'same' to map a code to itself, 'default' to catch all others.

🇬🇧 International mapping

Ry	Inbound Code(s)	Ry	Maps To	Ry	Note

🇺🇸 Domestic mapping

Ry	Inbound Code(s)	Ry	Maps To	Ry	Note

🏃 Run Reconciliation

Please upload at least the Inbound File and Coding Sheet to run.

Step 1 — Upload interface

←↻🏠🕒localhost:8501

Summarize🔍🌟⚙️🔖📄👤⋮

🏃 Run Reconciliation

Inbound: 2,454 rows | Coding Sheet: 125,001 rows | Outbound ⚠️ not uploaded | Error ⚠️ not uploaded | Retention ⚠️ not uploaded

Inbound Total (distinct)	→ International	→ Domestic	Output (distinct)
2,412	2,304	108	2,412

✅ Transaction ID check passed — Inbound and output both have 2,412 distinct sm_outgoing_transaction_id.

📊 Step 2: Summary

Total Records

4,781

✅ Exact Match

0

↑ 0.0%

⚠️ Error

0

🇬🇧 In Retention

0

🇺🇸 Transaction

591

🇺🇸 Outside Period

0

🇬🇧 Delta Reserve

0

🇬🇧 Unclassified

4,190

Treaty-Level Reconciliation Summary

Filter by Contract Name:

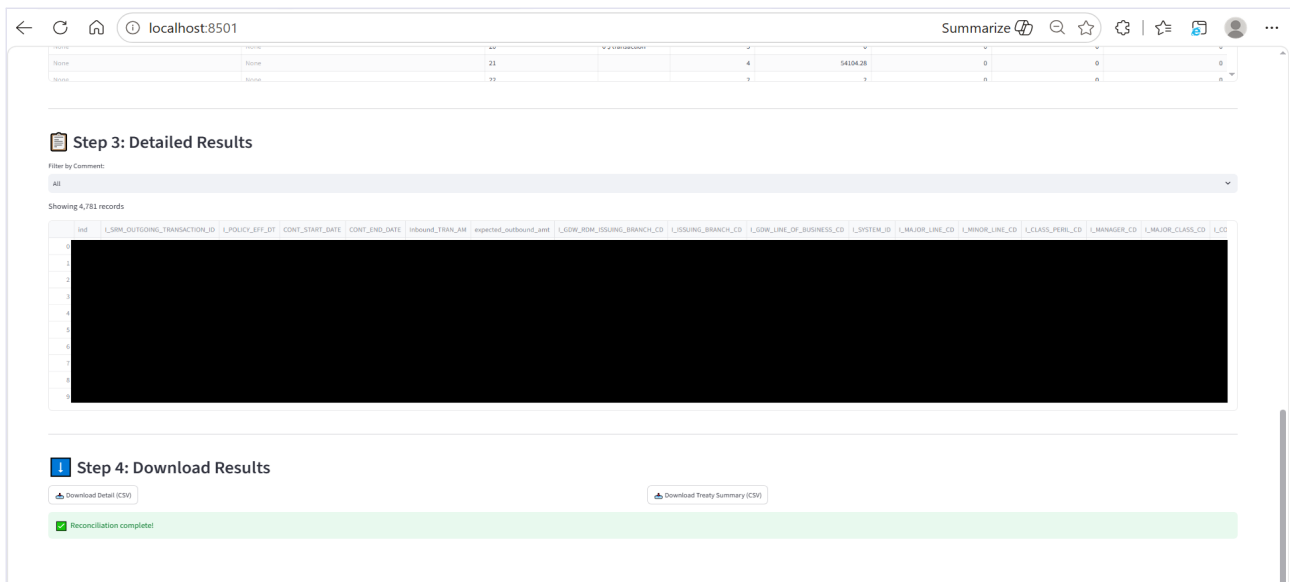
All

Filter by Comment:

All

EXT_SECTION_REF	CONTRACT_NAME	L_ISSUING_BRANCH_CD	Comment	Record_Count	Sum_Inbound_TRAN_AM	Sum_Expected_Outbound	Sum_Actual_Outbound	Variance (Expected - Actual)

Step 2 — Recon summary dashboard



Step 3 — Detailed results & download

5. Key Product Decisions

Decision 1 — SQL → PySpark → Streamlit: the full evolution

Recon began as manual SQL queries. Beyond requiring direct database access and technical execution, SQL introduced significant operational friction at scale — input files had to be uploaded to a database first, file format compatibility was inconsistent, large files took hours to load, and computation on high transaction volumes was slow.

PySpark eliminated the database dependency and handled large-scale data processing efficiently. But BAU analysts could not run it — command-line execution, environment dependencies, and PySpark setup created a hard technical barrier.

Streamlit eliminated the barrier entirely. A browser-based interface with file upload and click-based execution required zero technical prerequisites. BAU analysts could run a full recon cycle independently.

Decision 2 — Process-enforced validation gate, not system-enforced block

The utility operates as a process control gate — BAU runs recon first, reviews output, and only releases files to downstream systems after validation passes. A manual release step after automated validation gives BAU and delivery leads a documented checkpoint — a moment of explicit sign-off before billion-dollar financial data moves downstream.

Decision 3 — Treaty rules as single source of truth

Expected cession is calculated directly from the treaty report — not from separate configuration or hardcoded logic. This means the utility stays accurate as treaty terms change, without requiring code updates. The treaty report is the product's logic engine.

Decision 4 — Expanding output beyond mismatch flagging

Early versions only flagged session amount mismatches. Stakeholder feedback revealed BAU needed visibility into why transactions went to error or retention — not just that they did. Output was expanded based on observed behavior, not stated requirements. This came directly from the demo session with BAU and business stakeholders.

Decision 5 — Single file handling for international and domestic transactions

An early design limitation required separate files for each transaction type. Following stakeholder feedback during demo sessions, the utility was redesigned to handle both within a single input file — automatically identifying and applying the correct treaty rules per transaction type, eliminating a manual pre-processing step for BAU.

Decision 6 — Dynamic configurable parameters on the Streamlit screen

Certain recon parameters vary across production cycles and treaty types. Rather than hardcoding these values, key parameters were surfaced as configurable inputs directly on the Streamlit interface. BAU analysts can adjust parameters per cycle without any developer involvement.

Decision 7 — Recon summary surfaced directly on screen

A high-level recon summary is presented directly on the Streamlit screen at the end of each run — showing overall match status, transaction counts by category, and key mismatch flags at a glance. Detailed drill-down remains available in output files. This two-layer design matched how BAU and delivery leads actually consumed results: quick status check first, deep dive only when needed.

6. The Iteration Story

Technical correctness is not the same as product success. A tool that works perfectly but cannot be used by its intended users has failed its purpose.

V1 — SQL Scripts

Goal: Automate manual Excel recon

The first version replaced Excel with SQL queries — faster, more accurate, handling higher transaction volumes. But it required database access, SQL knowledge, and file uploads to the database were time-consuming with inconsistent format support. Computation on large datasets was slow.

- **What broke:** Data uploads to DB were time-consuming; file format sanitization was difficult; computation on large datasets was slow. BAU still could not run it without a data engineer.
- **What it revealed:** The real problem was not just recon accuracy. It was recon accessibility.

V2 — PySpark Script

Goal: Eliminate database dependency, handle scale

PySpark removed the database upload requirement entirely — files processed directly, handling millions of transactions efficiently. Technically the best version yet. But adoption failed for the same reason as V1.

- **What broke:** PySpark environment setup was a barrier; command-line execution was too technical for BAU users to run independently.
- **What it revealed:** The barrier was not the tool's logic — it was the tool's interface.

V3 — Streamlit Application

Goal: Full BAU self-sufficiency, zero technical prerequisites

Streamlit repackaged the same recon logic into a browser-based interface — file upload, configurable parameters, click to run, summary on screen, detailed output for download. No database. No command line. No environment setup. BAU analysts ran it independently from day one.

The problem to solve was no longer recon accuracy — that was solved in V1. The unsolved problem was making accurate recon accessible to the people who needed to run it.

Version	Stack	Problem Solved	Problem Revealed
V1	SQL	Recon accuracy	DB uploads slow; format issues; compute-heavy; BAU dependent
V2	PySpark	Scale and DB dependency	PySpark setup and CLI execution inaccessible to non-technical BA
V3	Streamlit + Pandas	Full BAU self-sufficiency	—

The requirements document told me what the tool needed to do. The users told me what the tool needed to be.

7. Stakeholder Collaboration

Building the utility was only half the work. The recon logic, output design, and interface decisions were shaped through active collaboration with three stakeholder groups.

Business Analysts — Defining the Logic

Working closely with the BA team, treaty parameters were mapped to expected cession calculations — confirming edge cases, clarifying how international and domestic transactions were handled differently under the same treaty, and validating that the utility's expected output matched what the business considered correct.

- Confirmed recon logic per treaty type
- Edge case handling for attachment criteria variations
- Correct treatment of international vs domestic transactions

BAU Operations Analysts — Shaping the Interface

Demo sessions revealed gaps not visible during build. Each became a product decision. None were anticipated during initial build.

- Single file handling for mixed transaction types
- Dynamic configurable parameters — no developer involvement required per cycle
- Two-layer output: summary on screen, detail in downloadable file

Product Owners / Delivery Leads — Building Confidence

The utility gave POs a structured, repeatable validation output — the same checks, the same logic, every cycle — replacing subjective manual review with automated, documented evidence of cession accuracy.

- Structured, repeatable validation evidence per production cycle
- Documented checkpoint before every downstream release decision
- Stakeholder confidence in the data gate

The requirements document told me what the tool needed to do. The users told me what the tool needed to be.

8. Outcomes

Metric	Before	After	Impact
Recon cycle time	2 working days	15 minutes	99.6% reduction
BAU technical dependency	QA team runs recon	BAU runs independently	Fully eliminated
Recon method	Manual Excel	Automated Streamlit app	Zero manual calculation
Release confidence	Subjective manual review	Automated documented evidence	Structured control
Tool adoption	Single project	Expanding firm-wide	Organic internal growth

For BAU Analysts

The 2-day manual recon cycle was replaced by a process any BAU analyst could run independently in 15 minutes. BAU analysts moved from being blocked by recon to being in control of it.

For QA / Data Engineers

Recon overflow work absorbing QA capacity every production cycle was eliminated. Technical resource was freed to focus on actual validation responsibilities.

For Product Owners / Delivery Leads

Every downstream release decision — covering millions of transactions and billions in financial impact — is now backed by automated, documented validation evidence rather than manual judgment.

For the Firm

Other reinsurance projects within the firm are adopting the utility without customisation. What started as a solution to one project's operational gap is becoming a reusable internal product.

99.6% reduction in recon cycle time — across a system processing millions of transactions covering billions in financial impact, every production cycle.

9. What I Would Do Differently

1. Involve BAU users from V1, not V3

The usability gap that killed V1 and V2 adoption was entirely predictable. If a BAU analyst had been observed during V1, the interface problem would have surfaced immediately.

What I'd do differently: Run a single user observation session with a BAU analyst before writing a line of V1 code.

2. Define success metrics before build, not after

The 99.6% cycle time reduction is compelling but was measured retrospectively. No formal baseline was captured at the start.

What I'd do differently: Document baseline explicitly before building: time per step, error rate, resource hours per cycle.

3. Structure demo sessions as formal feedback rounds

Demo sessions were run informally without a structured agenda. Some requirements emerged later than they should have.

What I'd do differently: Treat every demo as a structured feedback round with pre-defined questions and systematic observation.

4. Document product decisions in real time

Decisions were made and implemented without formal documentation of alternatives considered or reasoning applied.

What I'd do differently: Maintain a lightweight decision log throughout: what was considered, what was chosen, why.

10. PM Learnings

1. The user's constraint is part of the product requirement

V1 and V2 were technically correct but the user's ability to operate the tool is as much a requirement as the tool's logic. Every requirement has two dimensions: what the product does, and who can do it without help.

2. The real problem is rarely the stated problem

The stated problem was: recon is slow and inaccurate. V1 solved both. Adoption failed because the real problem was that recon required technical resource for an operational BAU task. Keep asking why until the root cause is clear.

3. Users show you what they need — they rarely tell you

Single file handling, configurable parameters, summary on screen — none were in any requirements document. All emerged from watching BAU analysts interact with the tool in demo sessions.

4. Iteration is not failure — premature handoff is

Each version answered a question the previous one couldn't. The failure was handing V2 to BAU before V3's question had been asked. Before releasing to intended users, ask: can this person use this without help?

5. A product that spreads organically has found its market

Other reinsurance projects adopted the utility without a mandate. Organic spread — users recommending it without being asked — is the strongest validation signal. Everything else can be manufactured. Organic spread cannot.

Details anonymized to protect client confidentiality. | Jayshree Thite | jayshreethite.career@gmail.com